

StockNet - Parallelized Neural Network for Stock Price Prediction Using High-Performance Computing Resources

Benjamin Zheng

Henry Odza

Prishita Ghanathe Krishna

Ronald Han

Toby Cowles

Abstract

In this project, we aim to design and implement a stock price predictor using the HPC cluster. Our approach will focus on the S&P 500 historical dataset, which provides a good source of financial data for model training and evaluation. We will begin by collecting and preprocessing this data to make sure it is clean, consistent, and suitable for modeling. Using the computational power of the Markov cluster, we will develop and train multiple neural network models to predict stock price movements and identify stocks likely to outperform market trends. These models will be designed to capture nonlinear relationships and temporal dependencies within the market.

1 Introduction

2 Datasets and Preprocessing

2.1 Datasets

Our analysis utilizes historical stock price data from the S&P 500 index, obtained from the Kaggle Stock Market Dataset. The dataset contains daily trading records for 500 stocks spanning multiple years, providing a comprehensive view of large-cap U.S. equities.

For each stock $s \in \{1, 2, \dots, S\}$ where S is the total number of stocks, we extract time series data consisting of:

$$\mathbf{x}_s^{(t)} = [L_t, O_t, V_t, H_t, C_t, A_t]^T \in \mathbb{R}^6$$

where for each trading day t :

- (1) L_t : Low price
- (2) O_t : Open price
- (3) V_t : Trading volume
- (4) H_t : High price
- (5) C_t : Close price
- (6) A_t : Adjusted close price

2.2 Preprocessing

To ensure temporal consistency across training samples, we impose a strict completeness requirement. For a stock s to be included in the training set, its data must satisfy:

$$\text{valid}(s) = \begin{cases} 1 & \text{if } |T_s| \geq T_{\min} \text{ and } \nexists t \in [1, T_s] : \mathbf{x}_s^{(t)} = \text{NaN} \\ 0 & \text{otherwise} \end{cases}$$

where T_s denotes the number of available time steps for stock s and $T_{\min} = 1000$ is our minimum sequence length requirement.

This filtering criterion serves two purposes:

- (1) **Temporal Consistency**: Ensures all stocks have equal sequence lengths for batch processing
- (2) **Data Integrity**: Eliminates stocks with missing values that would introduce bias

After applying this filter, our dataset reduces from 500 to S_{valid} stocks, where:

$$S_{\text{valid}} = \sum_{s=1}^S \text{valid}(s)$$

For each valid stock s , we extract the most recent $T = 1000$ consecutive trading days, creating a standardized input representation:

$$X_s = [\mathbf{x}_s^{(1)}, \mathbf{x}_s^{(2)}, \dots, \mathbf{x}_s^{(T)}] \in \mathbb{R}^{T \times 6}$$

This yields a corpus of stock sequences:

$$X = \{X_1, X_2, \dots, X_{S_{\text{valid}}}\}$$

where each X_s represents a complete 1000-day trading history for stock s .

Raw stock prices exhibit high variance across different equities due to fundamental value differences (e.g., penny stocks vs. high-value stocks). To stabilize training and ensure gradient flow, we apply feature-wise min-max normalization. For feature dimension $f \in \{1, 2, \dots, 6\}$, we first aggregate all values across stocks and time:

$$\mathbf{v}_f = \{X_{s,t,f} : s \in [1, S_{\text{valid}}], t \in [1, T]\}$$

The normalization transformation is then:

$$X_{s,t,f}^{\text{norm}} = \frac{X_{s,t,f} - \min(\mathbf{v}_f)}{\max(\mathbf{v}_f) - \min(\mathbf{v}_f)}$$

This maps all feature values to the range $[0, 1]$, ensuring:

$$X_{s,t,f}^{\text{norm}} \in [0, 1] \quad \forall s, t, f$$

All of this is done within a DataLoader object that is then passed onto the different models for use during training and evaluation.

3 Model Design and Implementation

3.1 Long Short Term Memory (LSTM) Network

One of the architectures we used to implement StockNet is the Long Short Term Memory (LSTM) network. LSTMs are a type of recurrent neural network (RNN) that are well-suited for sequence prediction problems due to their ability to capture long-term dependencies in data. In our implementation, we designed an LSTM network with multiple layers to effectively learn from the historical stock price data.

The complete architecture consists of:

- (1) A multi-layer LSTM encoder that processes historical price sequences
- (2) A fully connected output layer that maps LSTM hidden states to price predictions
- (3) An iterative forecasting mechanism that generates future price sequences based on previous predictions

Our implementation employs a stacked LSTM architecture with L layers, where $L = 2$ in the default configuration. For layer $\ell \in \{1, 2, \dots, L\}$, the hidden state at time t is denoted $h_t^{(\ell)}$. The output of layer $\ell - 1$ serves as input to layer ℓ :

$$h_t^{(\ell)} = \text{LSTM}^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}, c_{t-1}^{(\ell)})$$

where $h_t^{(0)} = x_t$ represents the input sequence. The hidden size is set to $h = 64$ neurons per layer.

To prevent overfitting, we apply dropout regularization between LSTM layers. During training, dropout randomly sets a fraction p of hidden state activations to zero:

$$h_t^{(\ell)} \leftarrow h_t^{(\ell)} \odot m^{(\ell)}$$

where $m^{(\ell)} \sim \text{Bernoulli}(1 - p)$ and $p = 0.2$ in our configuration.

At inference time, activations are scaled by $(1 - p)$ to maintain expected values. The LSTM's final hidden state $h_T^{(L)}$ at the last time step T is transformed to a scalar price prediction via a fully connected layer:

$$\hat{y}_{T+1} = W_{out} \cdot h_T^{(L)} + b_{out}$$

where $W_{out} \in \mathbb{R}^{1 \times h}$ and $b_{out} \in \mathbb{R}$ are learnable parameters. This yields the predicted normalized close price for the next time step. Raw stock price data undergoes feature-wise min-max normalization to stabilize training:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

where x_{min} and x_{max} are computed across all stocks and time steps in the training set. Normalized values lie in $[0, 1]$. We extract only the close price feature for autoregressive modeling, resulting in input sequences $X \in \mathbb{R}^{N \times (T-1) \times 1}$ where N is the number of stocks and $T = 1000$ is the sequence length.

For our loss function, we employ Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B (y_i - \hat{y}_i)^2$$

where B is the batch size, y_i is the true normalized close price, $\hat{y}_i$ is the prediction, and θ represents all model parameters. The main idea being that MSE penalizes larger errors, encouraging the model to produce more accurate price predictions.

The Adam optimizer was used with learning rate $\alpha = 0.001$:

$$\theta_{t+1} = \theta_t - \alpha \frac{m_t}{\sqrt{v_t} + \epsilon}$$

where m_t and v_t are exponentially weighted moving averages of gradients and squared gradients, respectively. The training process iterates for $E = 1000$ epochs with batch size $B = 32$. We shuffle training samples at each epoch to reduce overfitting to batch ordering.

As for evaluation, forecasting k steps ahead, we employ an iterative autoregressive approach. Starting with historical sequence $\mathbf{x} = [x_1, x_2, \dots, x_T]$, we predict:

- (1) $\hat{x}_{T+1} = f_{\theta}(\mathbf{x})$
- (2) $\hat{x}_{T+2} = f_{\theta}([x_2, x_3, \dots, x_T, \hat{x}_{T+1}])$
- (3) $\hat{x}_{T+k} = f_{\theta}([x_k, x_{k+1}, \dots, x_T, \hat{x}_{T+1}, \dots, \hat{x}_{T+k-1}])$

where f_{θ} represents the trained LSTM model. This sliding window approach maintains constant input sequence length while incorporating previously generated predictions.

Final predictions are transformed back to original price scale:

$$\hat{y}_{price} = \hat{y}_{norm} \times (x_{max} - x_{min}) + x_{min}$$

This allows direct interpretation of forecasted prices in currency units and facilitates comparison with actual market values.

3.2 Convolutional Neural Network (CNN)

4 Parallelization Strategy

5 Experiments

6 Results